

SAE 2.02

Projet Démineur

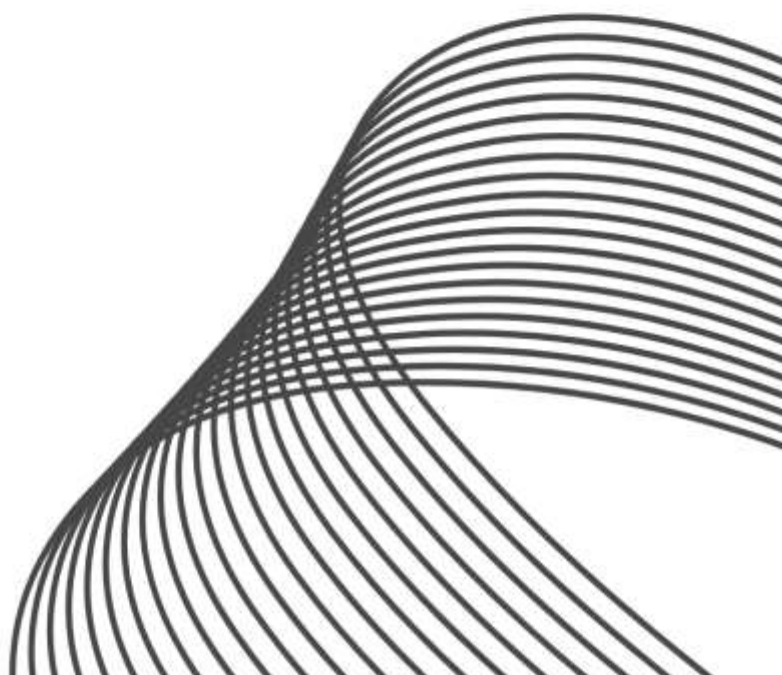
ANNÉES 2022-2023

RÉALISÉ PAR:

**ANEBAJAGAN ERIC &
YASSINE JABOU**

POUR:

MR. BOUCENNA



SOMMAIRE :

Introduction et Pré-Rflexion :	3
Organisation du code du Démineur :	4
La Fonction « Affichage » :	5
La fonction « GenereBombe »	7
La fonction « CaseAdjacente » :	8
La Fonction « SaisieJ1 » :	9
La Fonction « Saisie aleatoire » :	10
La Fonction « Sauvegarde » :	12
La Fonction « Couleur » :	13
Perspective et Conclusion :	14

Présentation :

Le présent rapport documente le projet de développement d'une interface de démineur réalisé dans le cadre de notre formation en BUT GEII (Génie Électrique et Informatique Industrielle). Ce projet nous a permis d'explorer les concepts fondamentaux de la programmation en langage C, ainsi que de mettre en pratique nos connaissances acquises tout au long de notre parcours.

Dans ce rapport, nous allons présenter les différentes étapes de développement de notre interface de démineur, depuis la phase de conception jusqu'à la réalisation finale du jeu. Nous décrirons les choix techniques que nous avons faits, les difficultés rencontrées ainsi que les solutions mises en œuvre pour les surmonter.

L'objectif :

Le projet vise à développer une interface de jeu pour le démineur. Le démineur est un jeu solo qui se déroule sur une grille rectangulaire. Certaines cases de la grille contiennent des mines, tandis que d'autres sont vides. Initialement, les contenus des cases sont cachés (couverte) et l'objectif est de découvrir quelles cases contiennent des mines.

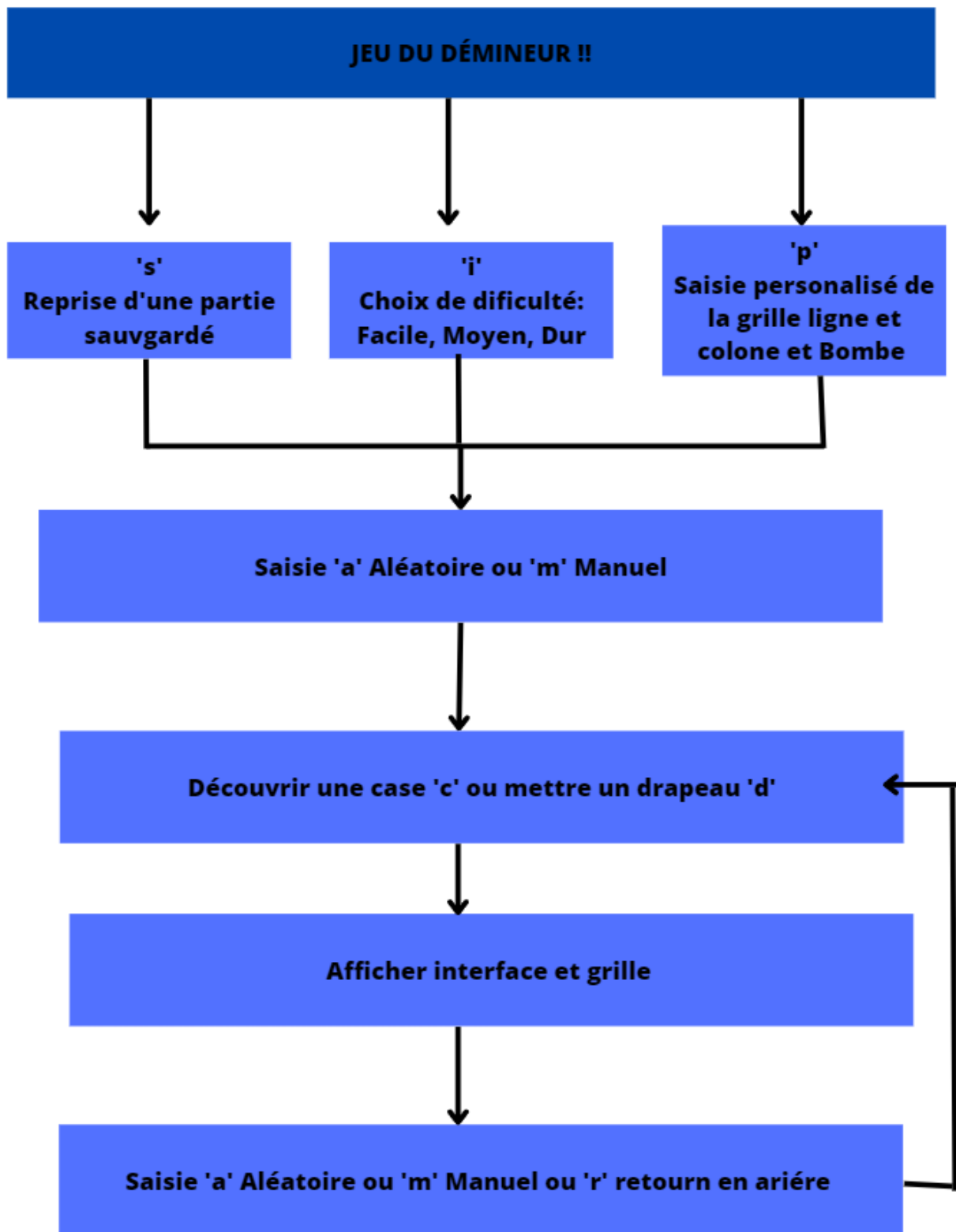
Le jeu se déroule comme suit : le joueur sélectionne une case encore couverte pour révéler son contenu. Si la case contient une mine, la partie se termine et le joueur perd. Si la case est vide, elle affiche le nombre de mines présentes dans les cases adjacentes, puis la partie continue. La partie se termine lorsque le joueur a découvert toutes les cases.

Les cases adjacentes font référence aux cases qui touchent la case en question par un côté ou un coin. Une case située au centre de la grille a donc 8 cases adjacentes. Les cases le long des bords en ont 5 et celles aux coins en ont 3.

Pré-Réflexion :

Pour préparer notre projet, nous avons réfléchi au fonctionnement global de notre programme. Nous avons décidé de créer un tableau invisible de type `"int"` dans lequel nous placerons les bombes (représentées par 0) et les valeurs des cases adjacentes. Parallèlement, nous avons créé un tableau visible de type `"char"` initialisé à 'X' qui sera affiché au joueur. Lorsque l'utilisateur saisit des coordonnées, le programme affiche la valeur correspondante en fonction de ces coordonnées dans le tableau visible.

ORGANISATION DU CODE DU DEMINEUR :



LA FONCTION « AFFICHAGE » :

Nom de la fonction : "Affinterface"

Rôle : Permet d'afficher à l'utilisateur les règles du jeu

Explication :

Nous avons travaillé sur la conception d'une interface simple pour le démineur, en commençant par des croquis sur papier. Par la suite, nous avons procédé à son implémentation dans une fonction appelée "Affiche Interface" dans notre programme, en utilisant des instructions "printf" pour afficher les éléments graphiques de l'interface. Nous avons placé la fonction "Affiche Interface" après chaque saisie pour actualiser l'interface du jeu. Pour apporter plus de dynamisme, nous avons ajouté la fonction système "Clear" qui permet de supprimer les grilles précédentes affichées dans le terminal.

Photo du programme :

```
#####
#
#           Jeu du demineur!
#
#####
# Règle du jeu :
#
# 1) CONFIG:  i:  Initiale           p:  Personnaliser
#
# 2) SAISIE:  a:  ALEATOIR           m : Manuel
#
# 3) REVEAL:  d:  DRAPEAU            c : CASE
#
# 3) SYSTEME: s:  SAVE               r : retourn en arrière
#
#####
```

Nom de la fonction : "CreateTAB"

Rôle : Créer le tableau où l'on placera les Bombes et le tableau que le joueur verra

Explication :

Avec la taille de la grille souhaitée nous procédons à l'initialisation d'un tableau de type "char" et "int". Nous utilisons la fonction "CreateTAB" pour créer le tableau, où nous attribuons la valeur -1 aux éléments du tableau de type "int" et 'X' aux éléments du tableau de type "char". Cette initialisation prépare la grille du démineur en assignant des valeurs par défaut à chaque case.

Nom de la fonction : "AffTAB"

Rôle : Permet d'afficher la Grille du Joueur

Explication :

Pour une meilleure organisation, nous avons séparé l'interface en deux parties distinctes : les règles du jeu et les scores avec la grille. Cela permet d'avoir une interface présente dès le début du jeu. Après chaque saisie, nous appelons d'abord la fonction "Affiche Interface" pour afficher les règles et les scores, puis nous appelons la fonction "Affiche Grille" pour afficher la ligne des scores ainsi que la grille. Cela est réalisé à l'aide de boucles "for" imbriquées pour parcourir les lignes et les colonnes de la grille.

Photo du programme :

```
#####
#      Case a ouvrir: 34      Nombre de Bombe: 15      #
#####

      |0|      |1|      |2|      |3|      |4|      |5|      |6| | | | |
|0|      | X |      | X |      | X |      | X |      | X |      | X |      | X |      |0|
|1|      | X |      | X |      | X |      | X |      | X |      | X |      | X |      |1|
|2|      | X |      | X |      | X |      | X |      | X |      | X |      | X |      |2|
|3|      | X |      | X |      | X |      | X |      | X |      | X |      | X |      |3|
|4|      | X |      | X |      | X |      | X |      | X |      | X |      | X |      |4|
|5|      | X |      | X |      | X |      | X |      | X |      | X |      | X |      |5|
|6|      | X |      | X |      | X |      | X |      | X |      | X |      | X |      |6|

      |0|      |1|      |2|      |3|      |4|      |5|      |6|
```

Nom de la fonction : "Difficulté"

Rôle : Permet à l'utilisateur de choisir une grille prédéfinie

Explication :

Cette fonction est implémentée dans le `main ()` et au début du jeu le joueur choisira une version manuel ou initiale si il choisit initiale il aura le choix entre 3 grille de difficulté.

```
1)Tapez votre char : i
Niveau de difficulter 1:Facil 2:Moyen 3:Difficile █
```

LA FONCTION « GENEREBOMBE »

Nom de la fonction : "GenereBombe"

Rôle : Permet de placer les Bombes dans le tableau de INT

Explication :

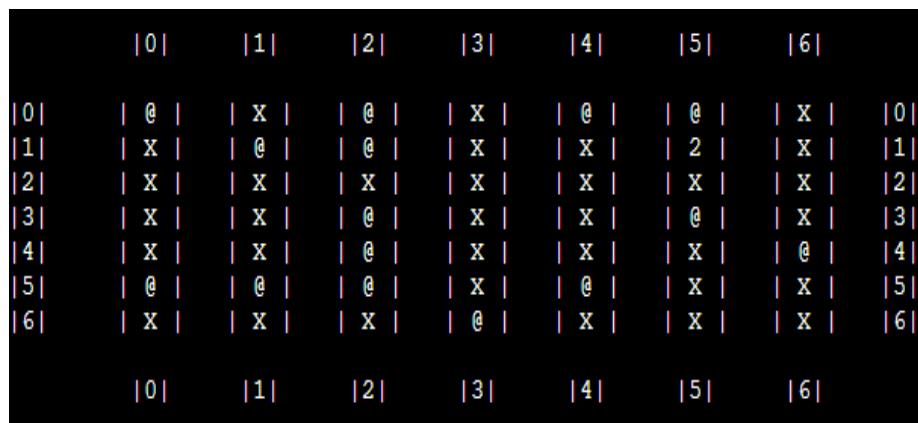
La fonction "Générer Bombe" utilise la fonction "srand()" pour initialiser la graine du générateur de nombres aléatoires. Ensuite, nous utilisons la fonction "rand() % max" pour générer un nombre aléatoire compris entre 0 et "max". Pour pouvoir utiliser ces fonctions, nous avons besoin d'inclure la bibliothèque <stdlib.h>.

Nous utilisons donc les variables "x" pour représenter la ligne et "y" pour représenter la colonne. Les bombes sont représentées par des zéros 0 dans le tableau de type "int" initialisé à -1. Nous ajoutons une condition qui permet de placer la bombe uniquement si la case correspondante dans le tableau de type "int" est égale à -1, car -1 indique qu'il n'y a pas de bombe à cet emplacement. Quand on trouve une bombe on affiche "GAME OVER" et le tableau avec toutes les bombes.

Pseudo-code :

```
1  Fonction genererBombe(ligne, colonne, nbBombe, TabSYS)
2      Début
3          srand(time(NULL)) //initialisation du temp
4          Pour i allant de 0 à nbBombe-1
5              x <- rand() % ligne // Générer n°Ligne Aléatoire
6              y <- rand() % colonne // Générer n°Colonne Aléatoire
7              Si TabSYS[x][y] est égal à -1//si aucun Bombe n'a
                  déjà été placer
8                  TabSYS[x][y] <- 0
9              Sinon
10                 i <- i - 1
11             Fin si
12         Fin pour
13     Fin
```

Photo du programme :



	0	1	2	3	4	5	6
0	0	X	0	X	0	0	X
1	X	0	0	X	X	2	X
2	X	X	X	X	X	X	X
3	X	X	0	X	X	0	X
4	X	X	0	X	X	X	0
5	0	0	0	X	0	X	X
6	X	X	X	0	X	X	X
	0	1	2	3	4	5	6

LA FONCTION « CASEADJACENTE » :

Nom de la fonction : "Case adja"

Rôle : Permet de placer le nombre de bombe présente dans les 8 case autour d'une case sélectionnée

Explication :

La fonction "CaseAdja" sera utilisée après avoir placé les bombes dans le tableau. Elle consiste en une lecture du tableau de type "int" pour chaque case, à l'aide de boucles "for" imbriquées. Pour chaque case, nous utilisons huit conditions distinctes (un "if" pour chaque case adjacente) pour vérifier si les coordonnées adjacentes contiennent une bombe. Si une coordonnée adjacente est égale à 0 (représentant une bombe), nous incrémentons une variable "bombe". Ensuite, nous attribuons à la case en cours la valeur de "bombe". De plus, si le nombre de bombes adjacentes ("bombe") est égal à 0, nous attribuons à la case la valeur -1 pour indiquer qu'il n'y a pas de bombe autour.

Les 8 coordonné :

[i-1][j-1]	[i-1][j]	[i-1][j+1]
[i][j-1]	[i][j]	[i][j+1]
[i+1][j-1]	[i+1][j]	[i+1][j+1]

Pour les cases situées le long des bords de la grille, nous avons ajusté le calcul des cases adjacentes en utilisant une condition. Par exemple, si une case se trouve sur le bord supérieur, nous ne prenons en compte que les cases adjacentes en dessous et sur les côtés, afin d'éviter de sortir du tableau.

Pseudo-code :

```
1 SI (i >= 0 ET i <= ligne ET j >= 0 ET j <= cologne ET TabSYS[i][j] != 0) ALORS
2   nbBombe <- 0
3 FIN SI
```

```
1 Fonction caseajda(ligne, cologne, TabSYS, TabA2, life1, nbTour1,
2   nbBombe1, x, y)
3   Début
4     Pour i allant de 0 à ligne-1
5       Pour j allant de 0 à cologne-1
6         Si i >= 0 ET i < ligne ET j >= 0 ET j < cologne
7           ET TabSYS[i][j] != 0
8             nbBombe <- 0
9             //Les 8 Coordonées si ya une bombe nbBombe++
10            Si i+1 < ligne ET j-1 >= 0 ET TabSYS[i+1][j-1] est égal à 0
11              nbBombe <- nbBombe + 1
12            Fin si
13            //Les 8 Coordonées si ya une bombe nbBombe++
14            Si nbBombe est égal à 0
15              TabSYS[i][j] <- -1
16            Sinon
17              TabSYS[i][j] <- nbBombe
18
```

LA FONCTION « SAISIEJ1 » :

Nom de la fonction : "SaisieJ1"

Rôle : Permet d'afficher la valeur cachée derrière la coordonnée tapée par le joueur

Explication :

Cette fonction permet d'afficher la case sélectionnée par le joueur. Pour cela, nous avons les conditions suivantes :

Si la case sélectionnée par le joueur est égale à 0 dans le tableau de type "int", alors nous affichons le caractère '@' pour représenter une bombe, et nous affichons également le message "Game over". Cela indique que le joueur a découvert une bombe et a perdu la partie.

Pour les autres cas, si la valeur de la case est égale à 1 dans le tableau de type "int", nous affichons le caractère '1'. Pour les autres chiffres, nous avons des conditions similaires où nous affichons le caractère correspondant à la valeur de la case dans le tableau.

Ces conditions nous permettent d'afficher la représentation visuelle appropriée pour chaque case, qu'il s'agisse d'une bombe ou d'un chiffre indiquant le nombre de bombes adjacentes.

Pseudo-code :

```
1  Fonction saisiJ1(ligne, cologne, TabSYS, TabA2, life1, nbTour1,
   nbBombe1, x, y)
2  Début
3      Si TabSYS[x][y] est égal à 0
4          nbBombe1 <- nbBombe1 - 1
5          life1 <- life1 - 1
6          TabA2[x][y] <- '@' // BOMBE
7      Sinon si TabSYS[x][y] est égal à 1
8          TabA2[x][y] <- '1'
9      Sinon si TabSYS[x][y] est égal à 2
10         TabA2[x][y] <- '2'
11     Sinon si TabSYS[x][y] est égal à 3
12         TabA2[x][y] <- '3'
13     Sinon si TabSYS[x][y] est égal à 4
14         TabA2[x][y] <- '4'
15     Sinon si TabSYS[x][y] est égal à 5
16         TabA2[x][y] <- '5'
17     Sinon si TabSYS[x][y] est égal à -1
18         TabA2[x][y] <- '0'
19     Fin si
20 Fin
```

LA FONCTION « SAISIE ALEATOIRE » :

Nom de la fonction : "SaisieAléa"

Rôle 1 : Permet si l'utilisateur le souhaite de choisir un coordonné aléatoirement

Explication :

La fonction "saisieAlea" est utilisée dans la boucle de jeu lorsque l'utilisateur souhaite sélectionner une case aléatoirement. Lorsque l'utilisateur saisit "a" pour choisir cette option, nous utilisons la fonction "srand()" pour initialiser la graine du générateur de nombres aléatoires. Ensuite, nous utilisons la fonction "rand()" pour générer des coordonnées aléatoires.

Cependant, nous ajoutons une condition pour vérifier si la case correspondante aux coordonnées générées a déjà été ouverte. Si c'est le cas, nous générons de nouvelles coordonnées.

Ainsi, la fonction "saisieAlea" permet à l'utilisateur de choisir une case aléatoire sans risquer de sélectionner une case déjà ouverte.

Pseudo-code :

```
1 SI (choixJ2 == 'a' ET TabA2[x][y] == 'X') ALORS
2     x = rand() % ligne
3     y = rand() % cologne
4 SINON
5     x = rand() % ligne
6     y = rand() % cologne
7 FIN SI
```

Rôle 2 : Permet à l'utilisateur de saisir manuellement coordonner

Explication :

Si l'utilisateur souhaite saisir manuellement ses coordonnées, il devra entrer la lettre 'm'. Dans ce cas, nous lui demanderons de saisir les coordonnées de la case qu'il souhaite sélectionner. Ces coordonnées sont sous forme de ligne et de colonne.

Cette option permet à l'utilisateur de contrôler plus précisément le choix de la case en saisissant directement ses coordonnées, plutôt que de les laisser être générées aléatoirement. Ainsi, l'utilisateur a le choix entre la saisie aléatoire avec 'a' ou la saisie manuelle avec 'm' pour déterminer la case qu'il souhaite découvrir dans le jeu.

Pseudo-code :

```

1  SI (choixJ2 == 'm') ALORS
2      AFFICHER "\nSaisissez le numéro de ligne : "
3      LIRE x
4      AFFICHER "\nSaisissez le numéro de colonne : "
5      LIRE y
6  FIN SI

```

Rôle 3 : Permet à l'utilisateur de voir la valeur cachée derrière sa coordonnée

Explication :

Après que l'utilisateur ait saisi la case, il a le choix de soit découvrir cette case en entrant la lettre 'c' :

Si l'utilisateur saisit 'c', le programme vérifie le contenu de la case sélectionnée. S'il s'agit d'une bombe (représentée par 0 dans le tableau de type "int"), la partie se termine et le joueur perd. Le programme affiche alors un message approprié, tel que "Game over".

Si la case sélectionnée ne contient pas de bombe, le programme affiche le nombre de bombes adjacentes à cette case.

Pseudo-code :

```

1  SINON SI (choixJ3 == 'c') ALORS
2      saisiJ1(ligne, colonne, TabSYS, TabA2, life1, nbTour1, nbBombe1, x, y)
3      nbTour1 <- nbTour1 + 1
4  FIN SINON

```

Rôle 4 : Permet à l'utilisateur de placer un drapeau si il pense qu'il y a une bombe à cette case.

Explication :

En plus de découvrir une case avec la lettre 'c', l'utilisateur a également la possibilité de placer un drapeau sur une case en entrant la lettre 'd'.

Lorsque l'utilisateur choisit de placer un drapeau, cela indique qu'il suspecte que la case contient une bombe. Le nombre de bombes restantes diminue, même s'il n'y a pas réellement de bombe présente. Cela permet de garder une trace du nombre de bombes potentielles qui n'ont pas encore été découvertes.

De plus, lorsqu'une case est découverte (que ce soit par choix de l'utilisateur ou par une action aléatoire), le nombre de tours restants diminue.

Pseudo-code :

```

1  SI (choixJ3 == 'd') ALORS
2      TabA2[x][y] <- 'D'
3      nbBombe1 <- nbBombe1 - 1
4  FIN SI

```

LA FONCTION « SAUVEGARDE » :

Nom de la fonction : "Sauvegarde"

Rôle : Permet si l'utilisateur le souhaite de sauvegarder sa partie et de reprendre sa partie déjà sauvegarder

Explication :

Si l'utilisateur souhaite sauvegarder la partie pendant le jeu, il peut le faire en entrant la lettre 's'. Pour la reprendre, au début du lancement de jeu.

Lorsque l'utilisateur saisit 's', le programme exécute une fonction de sauvegarde qui enregistre l'état actuel du jeu, y compris la grille visible, la grille invisible, les cases découvertes, les scores et tout autre élément nécessaire pour reprendre la partie ultérieurement.

La sauvegarde permet à l'utilisateur de quitter le jeu en cours et de revenir plus tard pour reprendre à partir du même point. Pour la reprise ça sera une lecture de fichier.

Pseudo-code :

```
1  SI (choixJ2 == 's') ALORS
2
3      AFFICHER "Vous sauvegardez la partie, au revoir"
4
5      fichier <- ouvrir fichier "partie_sauvegardee.txt" en mode écriture
6      SI (fichier == NULL) ALORS
7          AFFICHER "Erreur lors de l'ouverture du fichier."
8          RETOURNER
9      FIN SI
10
11     ÉCRIRE "La taille" dans le fichier
12     ÉCRIRE ligne dans le fichier
13     ÉCRIRE cologne dans le fichier
14
15     // Sauvegarde de la grille Visible
16     ÉCRIRE "Grille du joueur :" dans le fichier
17     POUR i allant de 0 à ligne FAIRE
18         POUR j allant de 0 à cologne FAIRE
19             ÉCRIRE TabA2[i][j] dans le fichier
20         FIN POUR
21     ÉCRIRE une nouvelle ligne dans le fichier
22 FIN POUR
23
24 // Sauvegarde de la grille système
25 ÉCRIRE "Grille avec les bombes :" dans le fichier
26 POUR i allant de 0 à ligne FAIRE
27     POUR j allant de 0 à cologne FAIRE
28         ÉCRIRE TabSYS[i][j] dans le fichier
29     FIN POUR
30     ÉCRIRE une nouvelle ligne dans le fichier
31 FIN POUR
32
33 ÉCRIRE "Les Scores:" dans le fichier
34 ÉCRIRE "# Case a ouvrir: %d      Nombre de Bombe: %d      #" avec nbTour1 et nbBombe1 dans le fichier
35
36 fermer fichier
37
38 AFFICHER "La partie a été sauvegardée avec succès."
39 SORTIR DE LA BOUCLE
40 FIN SI
```

Rôle : Permet d'afficher le programme en couleur rajoutant un côté esthétique

Explication :

Pour mettre de la couleur dans mon programme on a besoin de la bibliothèque :

```
#include <ncurses.h>
```

On a initialiser les paire de couleur utiles tel que rouge sur fond noir pour l'interface on a dans le main mis un `initscr()` et `star_color()` et un `attron` et `attrof` et remplaçant les `printf` en `printw`.

Le code :

[illegible]

Photo du programme :

```
#####
#
#                               #
#                               #
#                               #
#####
# Règle du jeu :                               #
#                               #
# 1) CONFIG:  i: Initiale           p: Personnaliser       #
#                               #
# 2) SAISIE:   a: ALEATOIR          m : Manuel             #
#                               #
# 3) REVEAL:   d: DRAPEAU           c : CASE               #
#                               #
# 3) SYSTEME:  s: SAVE               r : retourn en arrière #
#                               #
#####
```

PERSPECTIVE ET CONCLUSION :

Le démineur que j'ai développé présente de nombreuses qualités qui améliorent l'expérience de jeu. Il se distingue par un placement aléatoire des bombes, un calcul précis des cases adjacentes, une interface claire et soignée, une saisie aléatoire et manuelle des cases, la possibilité de placer des drapeaux, la fonction de sauvegarde et l'implémentation de couleurs.

Cependant, il existe des améliorations possibles pour enrichir davantage le démineur. L'une de ces améliorations serait d'implémenter une fonctionnalité de retour en arrière, permettant aux joueurs d'annuler leurs actions précédentes. Cela leur donnerait plus de flexibilité et de contrôle sur leur progression dans le jeu.

Une autre amélioration envisageable serait de créer une interface graphique conviviale. Cela offrirait une expérience de jeu plus immersive en permettant aux joueurs de sélectionner les cases à l'aide de leur souris et en ajoutant des éléments visuels attrayants.

En conclusion, mon démineur possède déjà plusieurs qualités, telles que le calcul des cases adjacentes, le placement aléatoire des bombes, la saisie aléatoire, la manipulation des drapeaux et la lecture de fichiers pour la sauvegarde. Cependant, des améliorations potentielles, telles que le retour en arrière et une interface graphique améliorée, pourraient encore enrichir l'expérience de jeu.